

Peer to peer multidimensional overlays: Approximating complex structures

Olivier Beaumont, Anne-Marie Kermarrec, Etienne Rivière

► To cite this version:

Olivier Beaumont, Anne-Marie Kermarrec, Etienne Rivière. Peer to peer multidimensional overlays: Approximating complex structures. [Research Report] RR-6248, INRIA. 2007, pp.19. inria-00164667v2

HAL Id: inria-00164667

<https://hal.inria.fr/inria-00164667v2>

Submitted on 25 Jul 2007

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

***Peer to peer multidimensional overlays:
Approximating complex structures***

Olivier Beaumont — Anne-Marie Kermarrec — Étienne Rivière

N° 6248

Juillet 2007

Thème COM

 ***apport
de recherche***

Peer to peer multidimensional overlays: Approximating complex structures

Olivier Beaumont* , Anne-Marie Kermarrec* , Étienne Rivière*

Thème COM —Systèmes communicants
Projets Asap et Cépage

Rapport de recherche n° 6248 —Juillet 2007 — 16 pages

Abstract: Peer to peer overlay networks have proven to be a good support for storing and retrieving data in a fully decentralized way. A sound approach is to structure them in such a way that they reflect the structure of the application. Peers represent objects of the application so that neighbours in the peer to peer network are objects having similar characteristics from the application's point of view. Such structured peer to peer overlay networks provide a natural support for range queries. While some complex structures such as a Voronoï tessellation, where each peer is associated to a cell in the space, are clearly relevant to structure the objects, the associated cost to compute and maintain these structures is usually extremely high for dimensions larger than 2.

We argue that an approximation of a complex structure is enough to provide a native support of range queries. This stems from the fact that neighbours are important while the *exact* space partitioning associated to a given peer is not as crucial. In this paper we present the design, analysis and evaluation of RayNet, a loosely structured Voronoï-based overlay network. RayNet organizes peers in an approximation of a Voronoï tessellation in a fully decentralized way. It relies on a Monte-Carlo algorithm to estimate the size of a cell and on an epidemic protocol to discover neighbours. In order to ensure efficient (polylogarithmic) routing, RayNet is inspired from the Kleinberg's small world model where each peer gets connected to close neighbours (its approximate Voronoï neighbours in Raynet) and shortcuts, long range neighbours, implemented using an existing Kleinberg-like peer sampling.

Key-words: Peer-to-Peer, Gossip-based overlay construction, Self-organization

Contact author:
Étienne Rivière, IRISA/INRIA, Campus Universitaire de Beaulieu 35042 Rennes cedex France.
Tel (+33) 2 99 84 73 37. Fax (+33) 2 99 84 71 71.

* Authors were supported in part by French ANR "Masse de données" ALPAGE project.

Réseaux pair à pair multidimensionnels : construction par approximation de structures complexes

Résumé : Les réseaux logiques fondés sur le paradigme pair-à-pair ont été reconnus comme un support de choix pour le stockage et l'interrogation de grandes masses de données de manière répartie. Une approche pertinente pour la construction de tels réseaux est de les structurer de façon à ce qu'ils reflètent la structure de l'application dont ils sont le support. Ainsi, les pairs représentent des objets de l'application, de telle sorte que les objets *voisins* dans le réseau partagent des caractéristiques proches du point de vue de l'application. De tels réseaux logiques structurés permettent un support *naturel* des requêtes complexes, comme les requêtes au plus proche voisin ou encore les requêtes par plage de valeurs sur un ensemble d'attributs. Bien que des structures comme le complexe de Voronoï, où chaque pair est associé à une cellule de l'espace de désignation, sont clairement adaptées au problème considéré, les coûts associés à leur construction et à leur maintien se révèlent prohibitifs dès lors que la dimension de l'espace de désignation est supérieure à 2.

Dans ce rapport, nous soutenons l'idée qu'une structure complexe peut être approximée. Une telle approximation est suffisante pour permettre le support natif des requêtes complexes comme les requêtes par plage de valeurs. Ceci découle du fait que les relations de voisinage sont ce qui importe le plus pour la construction du réseau, tandis que le partitionnement en cellule n'est pas nécessaire tant que les relations de voisinage peuvent être décidées. Ce rapport présente la construction, l'analyse et une étude de performances expérimentales de RayNet, un réseau logique à structuration partielle, fondé sur le complexe de Voronoï. RayNet organise les pairs (les objets) dans une approximation du complexe de Voronoï, calculé de manière totalement répartie. À cette fin, il utilise un algorithme de Monte-Carlo qui estime la taille d'une cellule et un protocole épidémique pour la découverte des relations de voisinage. Afin de permettre un routage efficace (d'ordre polylogarithmique en la taille du réseau), RayNet s'inspire du modèle de réseau petit-monde de Kleinberg, où chaque pair est connecté à un ensemble de voisins proches ainsi qu'à un ou des voisin(s) longue distance, ou raccourcis. La construction de ces liens longs utilise un protocole épidémique existant, proposant un échantillonnage biaisé vers la répartition de Kleinberg et mis en œuvre de manière répartie.

Mots-clés : Pair à pair, Protocoles épidémiques, Auto-organisation

1 Introduction

1.1 Structure versus search expressiveness

Plethora of peer to peer overlay networks have been proposed in the past years to manage data collection at a large-scale. Peer to peer overlays organize peers in a logical network and are characterized by their underlying structure. As far as data management is concerned, they differentiate each other by the expressiveness and efficiency of the search functionality they support. The expressiveness of search relates to the way data can be accessed: (i) exact search is used to access data objects identified by a unique identifier; (ii) attribute-based search enables to access data using a set of attribute, value pairs; (iii) in range queries, the attribute values are specified for a given range (a special case of which is nearest neighbour queries, where the object that lie “nearer” to a target point, according to an application dependant metric, is retrieved). At one end of the spectrum lie unstructured overlays in which each peer gets connected to a set of arbitrary neighbours. Such networks rely on constrained flooding techniques to search for data [22]. This provides a way to implement all types of search but such approaches often suffer from lack of efficiency. A query may need to ultimately visit the whole network to ensure exhaustive results. Fully structured overlays lie at the other end of the spectrum. In such networks, peers are organized along a precise structure such as a ring. In DHT-based networks [21], each object gets associated to a given peer. Such networks provide an efficient support for a DHT functionality. However, their expressiveness is naturally limited by the exact-match interface they provide.

We argue that, in order to improve upon the efficiency of expressive queries, the structure of the peer to peer overlay should reflect the application’s one. Peers are then application objects and get connected to neighbours (*i.e.* sharing similar characteristics from the application point of view). Such a logical organization provides a natural support for nearest neighbours and range queries. Such peer to peer overlays then support *natively* complex queries. Examples of such approaches are : Sub-2-Sub [27] and Meghdoot [11] for content-based publish and subscribe or Skip-graph based overlays [1, 10]. Those structures are however sometimes extremely complex to maintain accurately. For example, maintaining a Voronoï tessellation as in [4] involves a high overhead when the dimension is greater than two [6], and is prone to high levels of calculation degeneracy.

1.2 Weakening the structure.

In this paper we argue that a loose structure is actually enough from the search perspective. What really matters is that each peer gets connected to carefully chosen neighbours, so that the graph can be exhaustively visited. The exact logical structure is not as crucial, provided that its estimation enables correct routing for all requests. In this paper, we propose a general approach based on a Monte-Carlo algorithm to approximate a complex structure, in order to build a loosely structured overlay network. More precisely, we propose an algorithm to approximate the size of Voronoï cells, upon which we build neighbourhood relations.

1.3 Contributions.

The contributions of this paper are the following. First, we propose a general approach based on a Monte-Carlo method to approximate the size of a Voronoï cell. Then we propose the design and evaluation of RayNet, a weakly structured overlay network, achieving an approximation of a Voronoï tessellation. Following the generic approximation method, each peer in RayNet relies on an epidemic-based protocol to discover its neighbours. Using such a protocol, the quality of the estimation gradually improves to eventually achieve a close approximation of a Voronoï tessellation. This protocol ensures that each peer gets connected to its Voronoï-like neighbours while avoiding the need to accurately compute the exact Voronoï cells, thus keeping the overall overhead low. Each peer in RayNet also maintains a set of long-range links (also called shortcuts) to implement a small-world topology. Efficient (poly-logarithmic) routing in RayNet is achieved by choosing the shortcuts according to a distribution advocated by Kleinberg in [18]. Both links are created by gossip-based protocols. Finally, we evaluate the performance of RayNet through simulations and investigate its performance both in terms of bootstrapping time and routing performance.

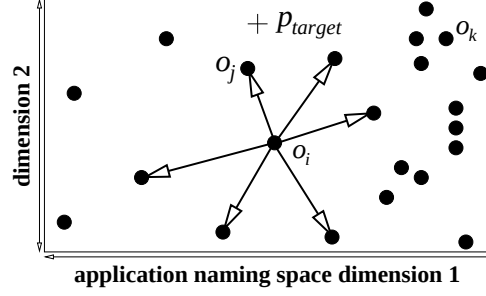


Figure 1: Illustration of the target structure (links are shown only for object o_i).

Note that implementing the query algorithm is actually out of the scope of this paper, and that we focus on the creation of the overlay itself.

2 Design rationale

2.1 System model.

We consider a system composed of n nodes, and a set of objects. We assume that each object is stored on the node that has created it. For the sake of clarity, we assume that there is a one to one mapping between an object and a physical node and denote such a pair as a peer in the rest of this paper¹. Application objects themselves are linked rather than computing entities. This design choice is similar to the one made for Skip-Graphs based systems [1, 10].

We consider a d dimensional attribute space. Each object is exactly identified by a value for each attribute². The attribute values of an object represent the virtual coordinates of the object in the attribute space. This may obviously lead to skewed distribution of objects in the naming space.

We assume that each peer maintains a partial view of the network, called its *view* and consisting of a list of neighbours (IP addresses and coordinates).

2.2 Structuring the network using Voronoi diagrams

Figure 1 describes coarsely the targeted structure for a two dimensional data set. A set of objects (black points) are maintained in the distributed application naming space. To achieve a structure that permits nearest neighbour and range queries possibilities, peers having *close* attribute values should be linked to each other in the overlay. Figure 1 shows such links for a sample object o_i . Our general goal for the creation of these links is as follows: for any point p_{target} belonging to the application naming space, for a query that passes through an object o_i , either o_i is the nearest to p_{target} and is the solution, or o_i knows a peer o_j that is nearer to the destination. This property ensures that a greedy routing process always succeeds, since the distance to the destination point is reduced at each step during the query propagation process.

A structure that ensures this property is the Delaunay graph, which is the dual of the Voronoi diagram. The Voronoi diagram of a set of generators points $\{p \in \mathbb{R}^d\}$ is a tessellation of $\mathbb{R}^d$ into disjoint cells. Each cell $vc(p_x)$ is composed of all points that are closer to p_x than to any other generator in the set. The links we aim at creating are adjacency relations between objects cells, and compose the Delaunay graph.

We have already successfully used Voronoi diagrams in the context of routing mechanisms [4] in a structured object-to-object overlay. This overlay provides a native support for range queries and nearest neighbour queries for datasets over two dimensions naming spaces. However, maintaining accurately this structure is extremely costly when the dimension goes over 2 [6]. First, the number of neighbours an object needs to handle in an overlay of dimension > 2 is growing exponentially with the dimension. Second,

¹This can be easily extended, but allocation of objects to nodes or optimizations in presence of multiple objects per node are out of the scope of this paper.

²Relaxing this assumption is possible but out of the scope of this paper.

the maintenance cost to keep *exactly* all these links consistent in spite of nodes and links failure increases accordingly.

However, defining the exact Voronoï cells is more than what is actually needed to ensure that greedy routing succeeds in such a network. What matters is actually the fact that each peer gets connected to its “close” neighbours *long all directions*. Also, imposing a fixed size set of neighbours at each object is desirable for scalability and load balancing purposes.

We base our design on the following observation: *for an object o with neighbourhood consisting of objects whose Voronoï cell shares a boundary with o 's cell, the volume of o 's cell in the tessellation of all objects is the same as o 's cell volume in the Voronoï tessellation of only o and its neighbours*. We are thus interested in discovering neighbours (partial view of the network) $o.view$ for each object o in the system, for which the volume of o 's cell in the tessellation of $o \cup o.view$ is minimal. We use a fixed size set of neighbours, and each object exchanges its current view of the network by means of a gossip-based protocol. Figure 2 presents the principle of this evolution: the more peers an object detects, the more opportunities of choosing a peer configuration it encounters to improve its zone approximation. In the following section, we highlight the principles of gossip-based protocols used for overlay construction, presents the biased peer-sampling protocol we use to provide small world characteristics to the overlay (especially for routing efficiency purposes). We then describe the core of our protocol, that is gossip-based construction of *coverage and closeness* at each peer, and the mechanisms that permit this construction, Monte-Carlo Voronoï cell size estimation.

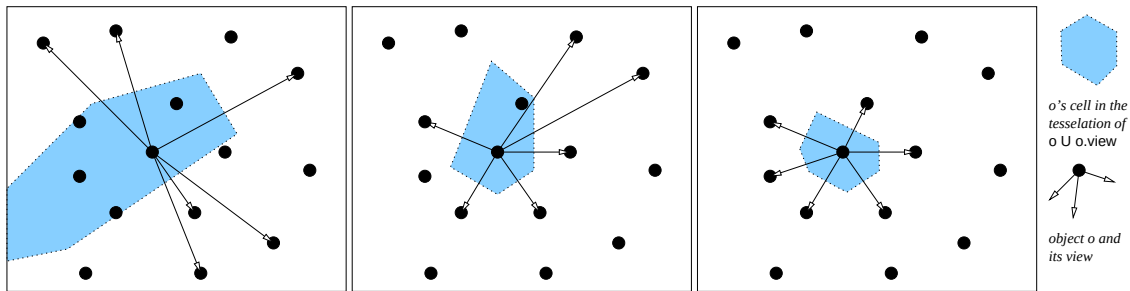


Figure 2: Desired evolution of an object’s neighbourhood: convergence towards the smallest (estimated) Voronoï cell. From random connections (left) to smallest possible zone (right).

3 Approximation through gossip

In this paper, we use gossip-based protocols to create and maintain the peer to peer overlay network. Although the focus of this paper is to approximate the neighbourhood at each peer, ultimately routing efficiently (in poly-logarithmic time) through the structure is an important concern. A small-world topology is created to achieve this. In this section we provide some background on small-world networks and gossip-based protocols. We then describe an existing gossip-based protocol that approximates a small-world topology. Finally, we present how we extend the generic gossip-based protocol framework to build the neighbourhood of each peer.

3.1 Small-world networks

Small-world network models were introduced to investigate the inherent routing capabilities of human relations networks. In such network models, each peer is connected to its closest neighbours in a topology as well as additional long-range contacts, also called shortcuts. Watts and Strogatz [28] introduce such a small-world topology where shortcuts are picked uniformly at random. In 2000, Kleinberg [18] demonstrated that poly-logarithmic routing could be achieved using a greedy algorithm if such shortcuts were chosen according to a specific distribution (d -harmonic). In his work, Kleinberg consider a $n \times n$ grid where every vertex has edges to its four direct neighbours and k (typically one) long-range neighbour(s).

This long-range neighbour is chosen with a probability proportional to $\frac{1}{l^d}$, where d is the dimension and l is the Euclidean distance between the vertex and its remote neighbour. These results can be extended to more general topologies and higher dimensions [3, 4, 8].

3.2 Gossip-based overlay construction

Gossip-based protocols, first introduced to reliably disseminate events in large systems, have now been recognized as a scalable and reliable basic building block to instantiate and maintain peer to peer overlay networks, and applications. Their scalability stems from their simplicity, their ability to capture system dynamics and the emergent properties they lead to. They have been successfully applied to a large number of settings from reliable broadcast [5] to overlay maintenance [9, 13, 24, 26], and from aggregation [16] to system size estimation [23] and are now turned into a generic and sound substrate for building and maintaining large-scale overlay networks [25].

A gossip-based protocol relies on a periodic exchange of information between peers. Such a period is called a *cycle*. Each peer keeps a (usually fixed-size) set of peers, called its *view*. Periodically, each peer picks a target from its view of the system, exchanges some information with it and processes the received information. If the information exchanged relates to neighbourhood, such a protocol creates an overlay network. We focus on such protocols in this paper. A gossip-based protocol is characterized by the three following parameters:

- **Peer selection policy:** each peer p_i chooses periodically a gossip target from its view $p_i.view$;
- **State exchanged:** the state exchanged between peers is membership information and consists of a list of peers (subset of their views);
- **State processing:** upon receipt of the list, the receiving peer merges the list of peers received with its own view to compose a new list of neighbours (new view).

It turns out that these parameters can be tuned so that the resulting graph exhibit properties which are extremely close to those of a random graph [9, 13, 26], providing a *Peer Sampling Service*: each peer's view contains a set of randomly drawn other peers from the network and this view changes at each cycle. More generally, it has been shown that arbitrary structures can be maintained this way, including fully structured peer to peer overlay networks [12, 24, 27].

For instance, it has been shown in [7] that the peer sampling protocol can be biased in order to approximate the distribution advocated by Kleinberg to improve routing in small-world networks. This can be achieved by simply adapting the *state processing* phase, to keep in the view, a set of peers that exhibits a Kleinberg-like long link length distribution. We use this protocol, called *small-world peer sampling* in the remaining of this document, as the substrate of our protocol, to achieve efficient routing.

3.3 Approximating the close neighbourhood: Coverage and closeness

It has been shown in [12, 26] that the same generic gossip protocol can be used to enable each peer to create links to its closest neighbours according to a given proximity metric. The peer selected to gossip with is then chosen as the closest from the view, and the state processing keeps the closest peers from the union of the local and received views. Such a clustering protocol is usually run concomitantly with a peer sampling service in order to ensure connectivity and to leave peers with the ability to cluster nodes³.

In this paper we propose to use a generalization of such a protocol to approximate the neighbourhood of a given peer. However, minimizing distances to each peer independently is not sufficient to ensure that the routing will succeed in all directions. Thus, instead of optimizing each item of the view independently, our approach is to decide on a new view as a whole. That means that, at each gossip cycle, set of peers are examined as *configurations* (potential new views) and not independently. To the best of our knowledge, this is the first time such an approach of generalization of gossip-based overlay construction protocols is proposed.

³Obviously non uniform topologies would be prone to create disconnected clusters otherwise.

We denote as the *utility* of a new configuration the metric that permits us to decide whether a configuration is better than the current view or not. This utility is the estimation of the Voronoï cell size, as decided by our Monte-Carlo estimation algorithm, as introduced in Section 4.1. This metric ensures that (1) closeness is achieved, which means that eventually a peer will get to know peers that are as close as possible to itself but (2) coverage is ensured, *i.e.* eventually each portion of the space surrounding a peer is covered by a neighbour, if such a peer exists in the system.

4 Protocol details

In this section we provide the details of building and maintaining RayNet. RayNet is based on a gossip-based approach: at each cycle, an object o chooses a gossip partner o_d from its current view (or a subset of its view) of the system to gossip with. After the state is exchanged, o then evaluates if there exists a new view (configuration of objects) that ensures a better coverage and closeness. The candidate configurations have thus to be considered as a whole, and peers objects can not be selected independently.

4.1 View evolution using Voronoï cell size estimation

4.1.1 Size of the view.

To ensure coverage and closeness, an object uses the estimated volume of its Voronoï cell based on its set of neighbours. Effectively, greedy routing succeeds if o knows neighbours in each possible direction (to get closer to any other target object) and close neighbours (when the target object is close to o).

If the volume of the Voronoï cell at o is minimal, then o knows Delaunay neighbours in any direction and if the volume of this cell is the smallest possible one, then these peers are among o 's closest neighbours. In general, $2d + 1$ neighbours are enough to get a bounded Voronoï cell. In order to keep extra close neighbours, we set the size c of objects views to $c = 3d + 1$. Moreover, we assume that peers exchange their entire view during a gossip operation.

4.1.2 Monte-Carlo cell volume estimation.

Once views have been exchanged, object o needs to estimate the volume of its Voronoï cell, for every possible configuration (on a naive basis; we show in the following Sections that examining *all possible* configurations is not mandatory). The volume of the cell is computed for each configuration. That is, given a set of objects $o.view \cup o_d.view = \{o_1, \dots, o_n\}$, for each possible configuration $\{o_{i_1}, \dots, o_{i_c}\}$ of size c , we estimate the volume of the Voronoï cell of o in the tessellation of points $o \cup \{o_{i_1}, \dots, o_{i_c}\}$. Then, if a new configuration is found, for which the volume of the cell of object o is reduced, this configuration is used as o 's new view.

There is no need to effectively compute the cell itself, which would be computationally expensive and prone to high levels of calculation degeneracy. Instead, we propose a new Monte-Carlo method for estimating this volume. Figure 3 presents an illustration of this approach in a two dimensional space. Note that this approach scales to higher dimensions.

A set of R rays is created, whose starting point is o and directions are drawn uniformly at random on the unit hyper-sphere. To this end, we use the method described in [19] that provides uniform probability distribution of points on the hyper-sphere. Algorithm 1.left describes the method. Rays (dashed lines starting from o on Figure 3) will act as *probes*, for which we discover the closest intersection point p_{int} lying on the ray r with a (virtual) Voronoï cell of another object in the configuration, this object being the object o_2 for which $\lambda = \|p_{int}, o\| = \|p_{int}, o_2\|$ is minimal. For this, the function `compDistOnRay()` in Algorithm 1.left computes λ for each point. Distances $\lambda = \|p_{int}, o_2\|$ are represented by discontinuous lines from o_2 to the intersection p_{int} on Figure 3. Lines (a) to (b) of Algorithm 1.left present the selection of the closest peers for each ray. We keep all λ values for each ray (set Λ), and use them to compute the estimation of the cell volume as follows (line (c) of Algorithm 1.left). Each ray r is associated to a *ball* of radius λ_r whose volume is given by $(BallVol \times (\lambda_r)^d)/R$, where $BallVol$ is the volume of the unit ball in dimension d . The volume of the estimated cell is the average value, for all rays, of volumes of such balls (the contribution for each ray is represented as grey cones on Figure 3). Such an estimator of the volume of

the Voronoï cell is clearly unbiased, so that the estimated volume converges to the volume of the Voronoï cell when $R \rightarrow +\infty$. Nevertheless, the convergence strongly depends on the shape of the Voronoï cell, thus imposing the use of a large enough R (1.000 in the current implementation).

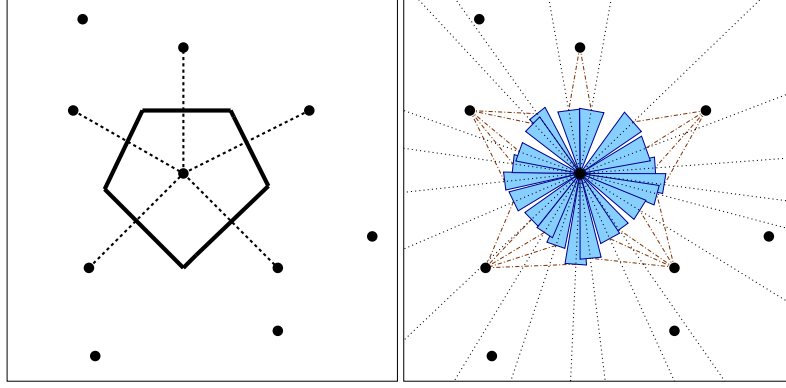


Figure 3: Illustration of the Monte Carlo method (o is the central point).

```

calcVolume()
parameters: config (SET[objects])
begin
  SET[double]  $\Lambda \leftarrow \emptyset$ 
   $o.rays \leftarrow createRays(R)$ 
  (a) for double[]  $r \in o.rays$  do
    double  $\lambda \leftarrow \infty$ 
    for object  $o_j \in config$  do
      double  $l \leftarrow compDistOnRay(r, o_j)$ 
      if  $l < dist$  then
         $\lambda \leftarrow l$ 
    (b)  $\Lambda \leftarrow \Lambda \cup \lambda$ 
    /* BallVol contains the unit Ball volume in
    dimension  $d$  */
  (c) return  $\frac{BallVol \times \sum_{\lambda \in \Lambda} (\lambda^d)}{R}$ 
end

update_naive()
parameters:  $o_d.view$  (SET[objects])
Local variables:
   $S$  : SET[objects]
  vol : double
begin
   $o.current\_vol \leftarrow calcVolume(o.view)$ 
  foreach  $S \in \mathcal{P}_c(view \cup o_d.view)$  do
    vol  $\leftarrow calcVolume(S)$ 
    if  $vol < o.current\_vol$  then
       $o.view \leftarrow S$ 
       $o.current\_vol \leftarrow vol$ 
  end

```

Algorithm 1: Monte-Carlo algorithm for estimating the volume of the cell for object o (left) and naive update algorithm for o receiving $o_d.view$ (right).

4.2 Discovery of a new configuration: naive approach

We describe in this section and in Algorithm 1.right the naive approach to select a new view for an object o upon reception of the view $o_d.view$. In order to determine the best view among the set of candidates, we need to estimate the volume of the Voronoï cell of o for the subgraph $S \cup o$ for each possible set S of c peer objects in the augmented view. That is, each possible subset of size c among $o.view \cup o_d.view$ shall be evaluated for replacement of $o.view$.

Evaluating all $C_{2c}^c = O(c!)$ possible configurations would provide exhaustive and accurate results, though at an unaffordable price. Therefore, we propose in the next Section a more realistic algorithm significantly reducing the overall complexity to a cost that is linear in the space dimension d .

4.3 Discovery of a new configuration: efficient, linear time approach

Algorithm 2 presented in this section requires rays for a given object to be chosen once and for all upon creation of the object, in order to save information between configurations' associated cell volumes. Each peer o maintains a bipartite graph $best$ containing on one side peers objects of $o.view$, and on the other side the rays $o.rays$. We denote by $best_O(r)$ the Voronoi neighbour o_p of o according to ray r : it is the node o_p such that a ray issued from o and whose direction is r first reaches the Voronoi cell of o_p (this entry is never empty). Similarly, we denote by $\{best_R(o_p)\}$ the set of rays for which o_p is the current Voronoi neighbour of o (this set may be empty).

The objective is as follows: to compute o 's new view, for each object o_p in $o_d.view \cap \overline{o.view}$ (i.e. all peers for which $\{best_R(o_p)\}$ does not contain any information), we determine the set of rays for which o_d is the Voronoi neighbour of o in the augmented view Voronoi diagram. This operation is described by lines (a) to (b) of Algorithm 2. Peers found to be a Voronoi neighbour of o for a given ray are stored in the set $improve$, which has the same semantic as $best_O$, except that entries for some rays can be empty.

On line (c), either $improve$ or $best_O$ has information, for each ray, about which peer in the augmented view is a Voronoi neighbour of o . The next step is to compute to which extent each peer is needed in the new configuration. More precisely, given a peer o_x , we compute the volume of the cell of o with all peers but o_x (lines (c)-(d)). If the volume of the cell increases dramatically, that means that peer o_x was mandatory to ensure closeness and proximity. On the other hand, if the volume remains the same, then peer o_x has no contribution to coverage nor closeness.

Note that, unlike the naive method (Algorithms 1), it is not necessary to iterate through all peers of the tested configuration to find the peer with the smallest λ value. This information is usually contained in either $best_O$, if such a peer lies in $o.view$, or in $improve$, if such a peer is a candidate peer from the distant view. The only case when one needs to iterate through all peers is when the best known peer for a given ray is o_x , the currently ignored peer.

Volumes associated to each peer (i.e. the volume without that peer in the configuration) are stored in the map $volumes$. This map is then sorted by decreasing volume values : starting from entries of peers that contributes highly to coverage and closeness, to entries of peers that have no or few contribution to coverage and closeness. The new configuration is built from the c peers that presents the maximum contribution, i.e. peers of the first c entries of $volume$.

The cost of the approach is as follows: there are up to $(r \times c)$ calls to method `distOnRay()`, if all c candidates were unknown to the current peer, and up to $(2 \times c)$ calls to `calcVolume()`. Each call to `distOnRay()` has cost 1: it is a fixed size set of scalar products. Each call to `calcVolume()` takes $r \times (1 + \frac{2 \times c - 1}{2 \times c})$ operations, where the term $\frac{2 \times c - 1}{2 \times c}$ stands for the few cases where the "best" peer is the currently ignored peer o_x (on average, $\frac{1}{2 \times c}$ occurrences per call). The overall cost is thus $\simeq 5(r \times c)$ operations. r is a constant. c depends only on the dimension of the naming space d , i.e. $c = O(d)$. The overall cost of the improved update algorithm is thus $O(d)$ operations.

5 Experimental evaluation

In this section, we evaluate RayNet along two metrics: (1) the time needed by a chaotic system to converge towards an overlay where all routes succeed and (2) when such an overlay is created, how many steps are required by greedy routing from any object to the nearest object of a target point, as a function of system size. Expected results are respectively: (1) a fast convergence and self-organization towards full success for routing requests and (2) a poly-logarithmic evolution of the route size according to the size of the system, thanks to the small-world peer sampling layer.

We developed a simulator using Java, and ran simulations for populations of objects ranging from 500 to 7.000 objects. The dimension of the object naming space d is ranging from 2 to 6. All objects points are drawn uniformly at random in this space. For all experiments, $r = 1.000$ rays were used to estimate cell volumes, and $3 \times d + 1$ neighbours are kept at each object. At each cycle, two exchanges take place, one for the small-world peer sampling layer (8 peers out of 20 maintained peers are sent), the other for the coverage and closeness layer (exchange of views). Also, for the first two cycles, each peer selects randomly 10 peers

```

update()
parameters:  $o_p.view$  (SET[objects]) /* distant view */
Local variables:
  improve (map ray  $\rightarrow$  object) init  $\emptyset$  /*improve has the same semantic as  $best_O$ */
  volumes (list of pairs (object,volume)) init  $\emptyset$ 
begin
(a)   foreach ray  $r \in o.rays$  do
      double  $best_\lambda = \perp$ 
      object  $imp = \perp$ 
      foreach object  $o_j \in (o_d.view \cap \overline{o_i.view})$  do
           $\lambda \leftarrow \text{distOnRay}(r, o_j)$ 
          if  $\lambda < \begin{cases} best_O(r) & \text{if } best_\lambda = \perp \\ best_\lambda & \text{if } best_\lambda \neq \perp \end{cases}$  then
               $imp \leftarrow o_j$ 
               $best_\lambda = \lambda$ 
          if  $best_\lambda \neq \perp$  then
(b)        $\text{improve}[r] = imp$ 
(c)   foreach object  $o_x \in o.view \cup (o_d.view \cap \overline{o_i.view})$  do
(d)       volumes  $\leftarrow$  volumes  $\cup$ 
           pair( $o_x, \text{calcVolumeImproved}(best \cup improve, (o_d.view \cap \overline{o_i.view}) \setminus o_x)$ )
      sort volumes by decreasing volume
       $o.view \leftarrow \{volumes_1.o, \dots, volumes_c.o\}$ 
      update  $best_O$  and  $best_R$ 
end

```

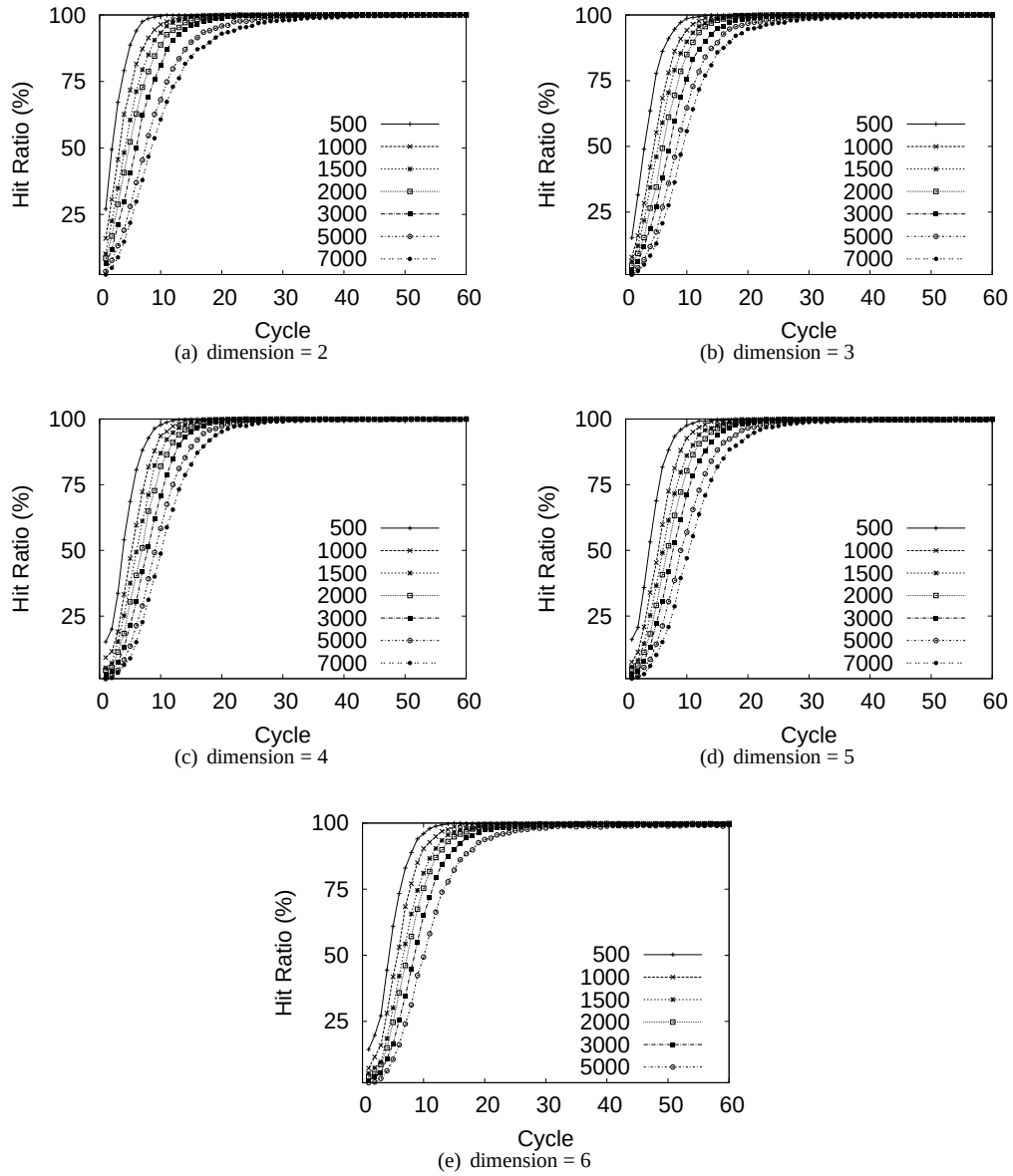
Algorithm 2: Update of object's view $o.view$: efficient approach. Sets $best_O$ and $best_R$ are constructed and coherent i.r.t. the current $o.view$ when starting the algorithm.

from the small-world peer sampling layer and assess them for potential inclusion in a new configuration to bootstrap the coverage and closeness level.

5.1 Bootstrapping the overlay.

First, we evaluate the time RayNet takes to converge towards an overlay state where every routing requests succeed. The overlay is initialized to a random graph for the small-world peer sampling layer, and no peer for the coverage and closeness layer. This makes sense as bootstrapping from a chaotic state is the worst case for gossip-based overlay construction mechanisms. More, following the proposal of [17] (with successful instantiations such as [14, 24, 27]), this represents the case where a distributed application needs the rapid instantiation of a routing substrate on top of a peer sampling layer. This experiment shows that our proposal fits perfectly in this scope, while being obviously applicable to long-term runs.

Figures 4 presents the results for all dimensions, and for different object population sizes. *Hit ratio* denotes the proportion of routes that succeed onto exactly the object that is nearest to the query destination. At each cycle, 20.000 random (object, destination point) pairs are tested. As expected, the hit ratio increases with the number of exchanges. In addition, perfect routing is achieved within at most 30 to 35 cycles, regardless of the dimension. Note that the cycle period is to be defined by the application, and depends on the trade-off between quality of service and cost on computing entities. It is possible however to bootstrap faster by using shorter periods at the beginning and to decrease it when steady state has been reached. In a dynamic scenario, objects would join gradually, and each object can use several short-term gossip exchanges to insert themselves faster in the overlay. The hit ratio converges slightly slower if there are more nodes. Note that this does impact neither the time a node would need to join an already constructed overlay, nor the complexity of local self-organization of the structure. Figures 4 show that approximating the structure does not impact routing correctness.

Figure 4: Evolution of routes *hit ratio* for dimensions 2 to 6.

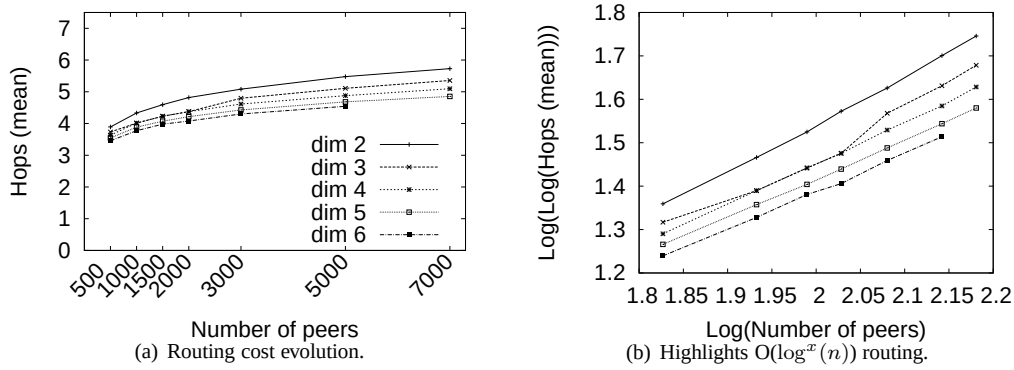


Figure 5: Routing efficiency (data for (a) and (b) is the same).

5.2 Routing efficiency.

The second evaluation metric is the routing efficiency: how many routing steps are needed on average to route between a source object and a destination point. This metric is directly impacted by the performance of the small-world peer sampling substrate as well as the quality of the close neighbourhood. It has a great impact on the efficiency of search mechanisms that can be proposed over the RayNet overlay. Figures 5.(a) and 5.(b) present the evolution of the routing costs as a function of the number of objects, for different dimensions. Particularly, Figure 5.(b) plots the $\log \log(\text{mean hops})$ as a function of $\log(\text{objects})$. The line shape of Figure 5.(b) proves that route sizes are poly-logarithmic in the number of objects, as expected by the small world characteristic of RayNet. We consider this property as being the key to scalability of future search mechanisms. The reason why higher dimensions present smaller routing paths is due to the fact that the size of the view at each objects increases linearly with the dimension d : for final steps (where small world links are not used), more possibilities are available for deciding on the next step of the route, which obviously slightly decreases the number of steps that use links from the coverage and closeness layer. This shows that approximating the structure does not impact routing performance.

Details of routing performance is given by Figure 6. For each dimension, route size evolution is given, along with the standard deviation. These Figures show that route size are balanced and that the distribution of routes lengths approaches a normal distribution. That means that no route is several orders of magnitude larger than another, which is a desirable behavior.

6 Related works

Other protocols were proposed to deal with multidimensional data querying and complex query support in large scale distributed systems. Structured overlays with exact-search interface have been used to implement range queries [2] even if such overlays are not *natively* addressing such capabilities. These approaches present relatively high costs of maintenance of the structures: either a second indexing mechanism based on objects rather than on nodes is built, whose cost is added to the cost of the structured overlay itself, or a single index is used but with the need for an implicit load balancing algorithm, to replace the inherent load balancing provided by hash mechanisms. RayNet steps away from this approaches by being designed with the native support for complex queries in mind from the beginning.

The authors used a similar approach to the one presented in this paper for the design of VoroNet [4]. This structured overlay organizes objects in an overlay that, like RayNet, reflects exactly the application semantic space, by using the Delaunay graph as the basic routing substrate and explicit small-world construction. Nonetheless, this approach suffers from two drawbacks: (1) maintaining the Delaunay complex for higher dimensions would be too costly (and, as presented in the Introduction, too much for the desired service) and (2) maintenance in two dimensions in face of churn is a difficult (yet not unsolvable) problem.

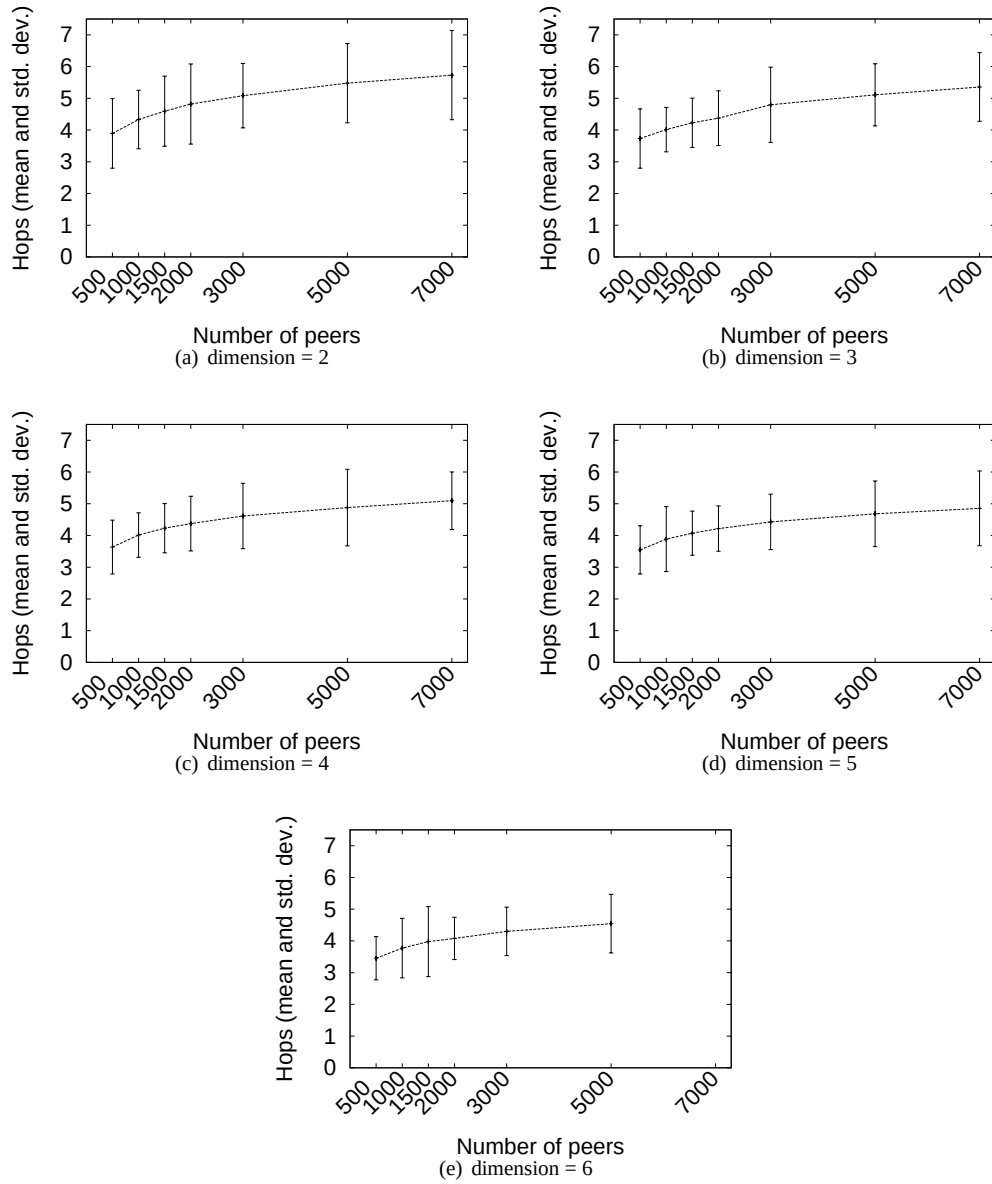


Figure 6: Evolution of routes size, details with standard deviations for dimensions 2 to 6.

RayNet addresses these two problems by (1) using an estimation of Voronoï cells as the basis for the construction of a subset of the Delaunay complex and (2) using Gossip-Based, self-organizing protocols that embed both protocol construction and re-organization in the same protocol, relieving the need for explicit fault tolerance mechanisms.

Skip-Webs [1] are multidimensional data structures that permit querying of data on a large scale, with multidimensional attributes. Nonetheless, maintaining such a structure in presence of churn may have a tremendous cost. Note that using Gossip-based techniques to construct this “Skip-List-like” structure could benefit from Gossip-based overlay construction protocols, such as the ones used for uni-dimensional data in GosSkip [10].

7 Conclusion

In this paper, we presented a new approach to create overlays that reflect a distributed application shared objects naming space. Organizing application objects in a distributed data structure based on the Delaunay graph of object points is sound but costly. We show that accuracy is not crucial and that reasonable approximation does not impact routing in such a structure. This paper presents the design and evaluation of RayNet, a peer to peer overlay that links objects in a multi-dimensional naming space, where each object’s view is drawn according to an estimation of its Voronoï cell size using a Monte-Carlo algorithm. Gossip-based protocols are extensively used to provide self-organization properties and routing efficiency. Simulation results convey the soundness and efficiency of the approach.

Next steps in this research are the following. First, we would like to investigate complex queries mechanisms for which RayNet was designed to be the support. At the moment, range queries are implemented by using constraint flooding ; refined mechanisms can be proposed by carrying some state on the query dissemination messages. We would like to investigate the scalability to higher dimensions of the mechanisms provided by [20]. Second, although gossip-based protocols are inherently resilient to nodes failures, few research has been done on securing such protocols. Following the early proposal of [15], we would like to investigate mechanisms to make our protocol resilient to adversary behaviours and detect malicious peers.

Acknowledgments We would like to thank François Bonnet, who helped us to integrate the gossip-based small-world peer sampling in RayNet [7] and Philippe Duchon, whose comments and expertise helped us on early stages of this work.

References

- [1] L. Arge, D. Eppstein, and M. T. Goodrich. Skip-webs: efficient distributed data structures for multi-dimensional data sets. In *PODC ’05: Proceedings of the twenty-fourth annual ACM SIGACT-SIGOPS symposium on Principles of distributed computing*, pages 69–76, Las Vegas, NV, USA, 2005.
- [2] J. Aspnes, J. Kirsch, and A. Krishnamurthy. Load balancing and locality in range-queriable data structures. In *PODC ’04: Proceedings of the twenty-third annual ACM symposium on Principles of distributed computing*, pages 115–124, St. John’s, Newfoundland, Canada, 2004.
- [3] L. Barrière, P. Fraigniaud, E. Kranakis, and D. Krizanc. Efficient routing in networks with long range contacts. In *DISC ’01: Proceedings of the 15th International Conference on Distributed Computing*, pages 270–284, London, UK, 2001. Springer-Verlag.
- [4] O. Beaumont, A.-M. Kermarrec, L. Marchal, and É. Rivière. VoroNet: A scalable object network based on voronoi tessellations. In *Proceedings of the 21st International Parallel and Distributed Processing Symposium (IPDPS 2007)*, Long Beach, CA, USA, March 2007.
- [5] K. P. Birman, M. Hayden, O. Ozkasap, Z. Xiao, M. Budiu, and Y. Minsky. Bimodal multicast. *ACM Transactions on Computer Systems*, 17(2):41–88, 1999.
- [6] J.-D. Boissonnat and M. Yvinec. *Algorithmic Geometry*. Cambridge University Press, 1998.

- [7] F. Bonnet, A.-M. Kermarrec, and M. Raynal. Small-world networks: Is there a mismatch between theory and practice? Technical Report inria-00155579, IRISA - ASAP, 2007.
- [8] P. Duchon, N. Hanusse, E. Lebhar, and N. Schabanel. Could any graph be turned into a small-world ? *Theoretical Computer Science*, 355(1):96–103, 2006.
- [9] P. T. Eugster, R. Guerraoui, S. B. Handurukande, P. Kouznetsov, and A.-M. Kermarrec. Lightweight probabilistic broadcast. *ACM Transactions on Computer Systems*, 21(4):341–374, 2003.
- [10] R. Guerraoui, S. B. Handurukande, K. Huguenin, A.-M. Kermarrec, F. L. Fessant, and É. Rivière. Gossip, an efficient, fault-tolerant and self organizing overlay using gossip-based construction and skip-lists principles. In *P2P '06: Proceedings of the Sixth IEEE International Conference on Peer-to-Peer Computing*, pages 12–22, Cambridge, UK, september 2006. IEEE Computer Society.
- [11] A. Gupta, O. D. Sahin, D. Agrawal, and A. E. Abbadi. Meghdoot: content-based publish/subscribe over p2p networks. In *Proceedings of the 5th ACM/IFIP/USENIX international conference on Middleware*, pages 254–273, New York, NY, USA, 2004. Springer-Verlag New York, Inc.
- [12] M. Jelasity and O. Babaoglu. T-man: Gossip-based overlay topology management. *Engineering Self-Organising Systems*, 1(15), 2005.
- [13] M. Jelasity, R. Guerraoui, A.-M. Kermarrec, and M. van Steen. The peer sampling service: experimental evaluation of unstructured gossip-based implementations. In *Middleware '04: Proceedings of the 5th ACM/IFIP/USENIX international conference on Middleware*, pages 79–98, New York, NY, USA, 2004. Springer-Verlag New York, Inc.
- [14] M. Jelasity and A.-M. Kermarrec. Ordered slicing of very large-scale overlay networks. In *P2P '06: Proceedings of the Sixth IEEE International Conference on Peer-to-Peer Computing*, pages 117–124, Cambridge, UK, september 2006. IEEE Computer Society.
- [15] M. Jelasity, A. Montresor, and O. Babaoglu. Towards secure epidemics: Detection and removal of malicious peers in epidemic-style protocols. Technical Report UBLCS-2003-14, University of Bologna, Department of Computer Science, Bologna, Italy, Nov. 2003.
- [16] M. Jelasity, A. Montresor, and O. Babaoglu. Gossip-based aggregation in large dynamic networks. *ACM Transactions on Computer Systems*, 23(3):219–252, 2005.
- [17] M. Jelasity, A. Montresor, and O. Babaoglu. The bootstrapping service. In *ICDCSW '06: Proceedings of the 26th IEEE International Conference Workshops on Distributed Computing Systems*, page 11, Lisboa, Portugal, July 2006. IEEE Computer Society.
- [18] J. Kleinberg. The small-world phenomenon: An algorithmic perspective. In *Proceedings of the 32nd ACM Symposium on Theory of Computing*, pages 163–170, Portland, OR, USA, may 2000.
- [19] D. E. Knuth. *Seminumerical Algorithms*, volume 2 of *The Art of Computer Programming*. Addison-Wesley, Reading, Massachusetts, Jan. 1981.
- [20] J. Liebeherr, M. Nahas, and W. Si. Application-layer multicast with delaunay triangulations. *IEEE Journal on Selected Areas in Communications, Special Issue on Network Support for Multicast Communication*, 40(8):1472–1488, 10 2002.
- [21] E. K. Lua, J. Crowcroft, M. Pias, R. Sharma, and S. Lim. A survey and comparison of peer-to-peer overlay network schemes. In *IEEE Communications survey and tutorial*, March 2004.
- [22] Q. Lv, P. Cao, E. Cohen, K. Li, and S. Shenker. Search and replication in unstructured peer-to-peer networks. In *ICS '02: Proceedings of the 16th international conference on Supercomputing*, pages 84–95, New York, NY, USA, 2002. ACM Press.

- [23] E. L. Merrer, A.-M. Kermarrec, and L. Massoulié. Peer to peer size estimation in large and dynamic networks: A comparative study. In *Proceedings of the 15th IEEE International Symposium on High Performance Distributed Computing*, pages 7–17, Paris, France, jun 2006.
- [24] A. Montresor, M. Jelasity, and O. Babaoglu. Chord on demand. In *P2P '05: Proceedings of the Fifth IEEE International Conference on Peer-to-Peer Computing (P2P'05)*, pages 87–94, Washington, DC, USA, 2005. IEEE Computer Society.
- [25] É. Rivière, R. Baldoni, H. Li, and J. Pereira. Compositional gossip: a conceptual architecture for designing gossip-based applications. *to appear in ACM SIGOPS Operating Systems Review*, 2007.
- [26] S. Voulgaris. *Epidemic-Based Self-Organization in Peer-to-Peer Systems*. PhD thesis, Vrije Universiteit, Amsterdam, November 2006.
- [27] S. Voulgaris, É. Rivière, A.-M. Kermarrec, and M. van Steen. Sub-2-sub: Self-organizing content-based publish and subscribe for dynamic and large scale collaborative networks. In *IPTPS'06: the fifth International Workshop on Peer-to-Peer Systems*, Santa Barbara, USA, FEB 2006.
- [28] D. J. Watts and S. H. Strogatz. Collective dynamics of small world networks. *Nature*, 393:440–442, 1998.

Contents

1	Introduction	3
1.1	Structure versus search expressiveness	3
1.2	Weakening the structure.	3
1.3	Contributions.	3
2	Design rationale	4
2.1	System model.	4
2.2	Structuring the network using Voronoï diagrams	4
3	Approximation through gossip	5
3.1	Small-world networks	5
3.2	Gossip-based overlay construction	6
3.3	Approximating the close neighbourhood: Coverage and closeness	6
4	Protocol details	7
4.1	View evolution using Voronoï cell size estimation	7
4.1.1	Size of the view.	7
4.1.2	Monte-Carlo cell volume estimation.	7
4.2	Discovery of a new configuration: naive approach	8
4.3	Discovery of a new configuration: efficient, linear time approach	9
5	Experimental evaluation	9
5.1	Bootstrapping the overlay.	10
5.2	Routing efficiency.	12
6	Related works	12
7	Conclusion	14



Unité de recherche INRIA Rennes
IRISA, Campus universitaire de Beaulieu - 35042 Rennes Cedex (France)

Unité de recherche INRIA Futurs : Parc Club Orsay Université - ZAC des Vignes
4, rue Jacques Monod - 91893 ORSAY Cedex (France)

Unité de recherche INRIA Lorraine : LORIA, Technopôle de Nancy-Brabois - Campus scientifique
615, rue du Jardin Botanique - BP 101 - 54602 Villers-lès-Nancy Cedex (France)

Unité de recherche INRIA Rhône-Alpes : 655, avenue de l'Europe - 38334 Montbonnot Saint-Ismier (France)

Unité de recherche INRIA Rocquencourt : Domaine de Voluceau - Rocquencourt - BP 105 - 78153 Le Chesnay Cedex (France)

Unité de recherche INRIA Sophia Antipolis : 2004, route des Lucioles - BP 93 - 06902 Sophia Antipolis Cedex (France)

Éditeur
INRIA - Domaine de Voluceau - Rocquencourt, BP 105 - 78153 Le Chesnay Cedex (France)
<http://www.inria.fr>
ISSN 0249-6399